

Date:04-10-2026

Lab # 05 TO UNDERSTAND AND OBSERVE THE SINGLY AND DOUBLY LINKED LIST BEAHVIOUR

Lab Objective:

- Comprehend the structural differences between singly and doubly linked lists, including memory requirements and traversal capabilities.
- Write and execute code to perform key operations (insert, delete, and traverse) in both singly and doubly linked lists.
- Identify scenarios where singly or doubly linked lists are preferred over arrays or other data structures due to their dynamic and non-contiguous memory allocation properties.
- Observe how linked lists handle insertion and deletion operations differently compared to other data structures, particularly arrays, and understand the time complexity involved.

Theory:

Singly Linked List:

A singly linked list is a linear data structure consisting of nodes, where each node contains a data part and a pointer (or reference) to the next node in the sequence. Unlike arrays, linked lists don't require contiguous memory locations, making them more flexible for dynamic memory allocation. However, due to its unidirectional nature, traversal is only possible from the head node to the end. Key operations include insertion, deletion, and traversal. Singly linked lists are useful when memory is fragmented or when elements need to be frequently added or removed without requiring memory reallocation.

Doubly Linked List:

A doubly linked list is similar to a singly linked list but with an added advantage: each node contains two pointers—one pointing to the next node and one to the previous node. This bidirectional structure allows traversal in both forward and backward directions. The doubly linked list's dual links make it easier to implement complex operations like deletion or reverse traversal but requires extra memory for storing an additional pointer in each node. Doubly linked lists are commonly used in applications that require flexible navigation, such as undo/redo functionality in software or in applications where bidirectional traversal is advantageous.

Singly Linked List

- **Definition:** A singly linked list is a data structure where each node contains data and a reference to the next node, forming a unidirectional chain.
- **Basic Operations:**
 - **Insert:** Adds a node at the beginning, end, or a specified position.
 - **Delete:** Removes a node from the beginning, end, or a specified position.
 - **Traverse:** Goes through each node in the list to access or display elements.

Code for Singly Linked List:

```
class Node:
```

Date:03-10-2026

```
def __init__(self, data):
    self.data = data
    self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    def delete_node(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            temp = None
            return
        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next
        if temp is None:
            return
        prev.next = temp.next
        temp = None

    def traverse(self):
        current = self.head
        while current:
            print(current.data, end=" ")
            current = current.next
            print()

# Example usage
sll = SinglyLinkedList()
sll.insert_at_end(1)
sll.insert_at_end(2)
sll.insert_at_end(3)
sll.traverse()
sll.delete_node(2)
sll.traverse()
```

Doubly Linked List

- **Definition:** A doubly linked list is a data structure where each node contains data and two references—one to the next node and one to the previous node—allowing traversal in both directions.

Date: 04-10-2026

- **Basic Operations:**

- **Insert:** Adds a node at the beginning, end, or a specified position.
- **Delete:** Removes a node from the beginning, end, or a specified position.
- **Traverse Forward/Backward:** Accesses elements in both forward and reverse directions.

Code for Doubly Linked List:

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node
        new_node.prev = last

    def delete_node(self, key):
        temp = self.head
        while temp:
            if temp.data == key:
                if temp.prev:
                    temp.prev.next = temp.next
                if temp.next:
                    temp.next.prev = temp.prev
                if temp == self.head:
                    self.head = temp.next
                temp = None
            temp = temp.next

    def traverse_forward(self):
        current = self.head
        while current:
            print(current.data, end=" ")
            current = current.next
        print()

    def traverse_backward(self):
        current = self.head
        while current and current.next:
            current = current.next

```

Date:03-10-2026

```
        current = current.next
while current:
    print(current.data, end=" ")
current = current.prev
print()
```

```
# Example usage dll =
DoublyLinkedList()
dll.insert_at_end(1)
dll.insert_at_end(2)
dll.insert_at_end(3)
dll.traverse_forward()
dll.delete_node(2)
dll.traverse_forward()
dll.traverse_backward()
```

Exercise:

1. Implement a singly linked list supporting essential operations such as insertion, deletion, and traversal to efficiently manage and explore data.
2. Design a doubly linked list with capabilities for insertion, deletion, and traversal in both forward and reverse directions, enabling flexible data navigation and management.
3. Examine and articulate the key differences between singly and doubly linked lists, including their structure, traversal capabilities, memory usage, and ideal use cases.

Date: 04-10-2026

Introduction:

A linked list is a linear data structure whose nodes are not stored in contiguous memory. A singly linked list node holds data and a next pointer, so traversal goes only forward. A doubly linked list node also holds a prev pointer, which allows forward and backward traversal. Because nodes are linked by pointers, insertion and deletion need no shifting of elements, unlike arrays.

Abstract:

This lab studies singly and doubly linked lists in Python. Both were implemented with insertion, deletion and traversal, and their structure, memory use and traversal ability were compared. The outputs confirmed that a doubly linked list supports traversal in both directions, at the cost of one extra pointer per node.

Procedure:

1. Opened a new notebook in Google Colab.
2. Created the `Node` class with `data` and `next` attributes for the singly linked list.
3. Created the `SinglyLinkedList` class with a `head` pointer set to `None`.
4. Wrote the insert methods to add a node at the beginning and at the end of the list.
5. Wrote the `delete\_node` method to remove a node by its value and the `traverse` method to print all values.
6. Inserted values, traversed the list, deleted one node and traversed again to check the output.
7. Created the `DNode` class with `data`, `next` and `prev` attributes for the doubly linked list.
8. Created the `DoublyLinkedList` class with insert and delete methods that update both `next` and `prev` pointers.
9. Wrote the `traverse\_forward` and `traverse\_backward` methods to print the list in both directions.
10. Ran the code and checked the forward and backward outputs.

Conclusion:

Both lists were implemented and tested successfully. A singly linked list is simpler and uses less memory, but it can only be traversed forward. A doubly linked list costs an extra pointer per node, but it allows bidirectional traversal and easier deletion. Compared to arrays, both lists insert and delete without shifting elements, though they lose direct indexing, so access by position takes $O(n)$.

Date:03-10-2026

Tasks:

Task 1:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    def delete_node(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            return
        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next
        if temp is None:
            print("Key not found")
            return
        prev.next = temp.next

    def traverse(self):
        current = self.head
        while current:
            print(current.data, end=" ")
            current = current.next
        print()
```

Date: 04-10-2026

```
sll = SinglyLinkedList()
sll.insert_at_end(1)
sll.insert_at_end(2)
sll.insert_at_end(3)
sll.insert_at_beginning(0)
sll.traverse()
sll.delete_node(2)
sll.traverse()

... 0 1 2 3
    0 1 3
```

Task 2:

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = DNode(data)
        new_node.next = self.head
        if self.head:
            self.head.prev = new_node
        self.head = new_node

    def insert_at_end(self, data):
        new_node = DNode(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
        new_node.prev = current

    def delete_node(self, key):
        current = self.head
        while current:
```

Date:03-10-2026

```
        if current.data == key:
            if current.prev:
                current.prev.next = current.next
            if current.next:
                current.next.prev = current.prev
            if current == self.head:
                self.head = current.next
            return
        current = current.next
    print("Key not found")

def traverse_forward(self):
    current = self.head
    while current:
        print(current.data, end=" ")
        current = current.next
    print()

def traverse_backward(self):
    current = self.head
    while current and current.next:
        current = current.next
    while current:
        print(current.data, end=" ")
        current = current.prev
    print()

dll = DoublyLinkedList()
dll.insert_at_end(7)
dll.insert_at_end(14)
dll.insert_at_end(21)
dll.insert_at_beginning(3)
dll.traverse_forward()
dll.delete_node(14)
dll.traverse_forward()
dll.traverse_backward()

... 3 7 14 21
     3 7 21
     21 7 3
```

Date:04-10-2026

Task 3:

Both are linear data structures made of nodes and both use non-contiguous memory. The difference is in the number of pointers stored in each node, which affects traversal, memory use and ease of operations.

Feature	Singly Linked List	Doubly Linked List
Node structure	data and next pointer	data, next and prev pointers
Traversal	Forward only	Forward and backward
Memory usage	Less (one pointer per node)	More (two pointers per node)
Deletion	Needs the previous node, found by traversing from head	Node can be unlinked directly using its prev pointer
Insertion at beginning	$O(1)$	$O(1)$
Insertion at end (no tail pointer)	$O(n)$	$O(n)$
Searching	$O(n)$	$O(n)$
Implementation	Simpler	More complex, more pointers to update
Common uses	Stacks, simple queues, memory limited cases	Undo/redo, browser history, playlists